

Disruption Budgets: Surviving Autoscaler Churn



Published on August 3, 2024



Webstack
Builders

Table of Contents

PDB Fundamentals	3
How PDBs Work	3
minAvailable vs maxUnavailable	5
Cluster Autoscaler Interaction	6
Scale-Down and PDBs	6
Avoiding Autoscaler Deadlocks	7
Node Rotation Strategies	8
Rolling Node Updates	8
Handling Stuck Drains	9
Production PDB Patterns	11
Recommended Configurations	11
Anti-Patterns to Avoid	13
Monitoring and Alerting	15
PDB Health Metrics	15
Health Thresholds	16
Conclusion	17



A PodDisruptionBudget (PDB (Pod Disruption Budget)) is a native Kubernetes resource that limits how many pods can be down simultaneously during voluntary disruptions – node drains, upgrades, autoscaler scale-downs. It's the mechanism that keeps your service available while the cluster changes around it. But poorly configured PDBs (Pod Disruption Budget) create a different problem: overly strict budgets can block node rotations entirely, prevent security patches, and cause cluster autoscaler deadlocks. A PDB (Pod Disruption Budget) is a contract between workload owners and cluster operators, and like any contract, the terms matter.

Here's a scenario I've seen more than once: a platform team schedules node rotation for security patching. The rotation begins at 2 AM. By 6 AM, the on-call engineer is paged – half the nodes are stuck in “draining” state.

Investigation reveals a critical service with `minAvailable: 3` and 3 replicas, but all 3 pods landed on nodes scheduled for rotation. The PDB (Pod Disruption Budget) that was supposed to protect availability is now **preventing** the security patch that protects availability. The lesson: PDBs (Pod Disruption Budget) require thinking about both directions – protecting from disruption **and** allowing necessary operations.

⚠ WARNING

A PDB (Pod Disruption Budget) that blocks all disruptions doesn't protect your service – it protects it from getting security patches, upgrades, and capacity optimization. The goal is controlled disruption, not zero disruption.

PDB Fundamentals

How PDBs Work

A PDB (Pod Disruption Budget) targets pods via a label selector and specifies either a minimum number that must remain available (`minAvailable`) or a maximum that can be unavailable (`maxUnavailable`). When something attempts to evict a pod – whether it's `kubectl drain` , the cluster autoscaler, or a node upgrade controller – the API (Application Programming Interface) server checks the PDB (Pod Disruption Budget) before allowing the eviction.

pdb-basics.yaml

```
1  apiVersion: policy/v1
```



```
2  kind: PodDisruptionBudget
3  metadata:
4    name: api-server-pdb
5    namespace: production
6  spec:
7    selector:
8      matchLabels:
9        app: api-server
10    # Minimum pods that must remain available
11    minAvailable: 2
12    # OR maximum pods that can be unavailable (pick one, not both)
13    # maxUnavailable: 1
```

Basic PDB (Pod Disruption Budget) structure targeting pods with the `app: api-server` label.

The PDB (Pod Disruption Budget) controller continuously tracks how many matching pods are healthy and calculates a key field: `disruptionsAllowed`. This number tells you how many pods can currently be evicted without violating the budget. If it's zero, nothing can be evicted. You can check the current state with `kubect1 get pdb`:

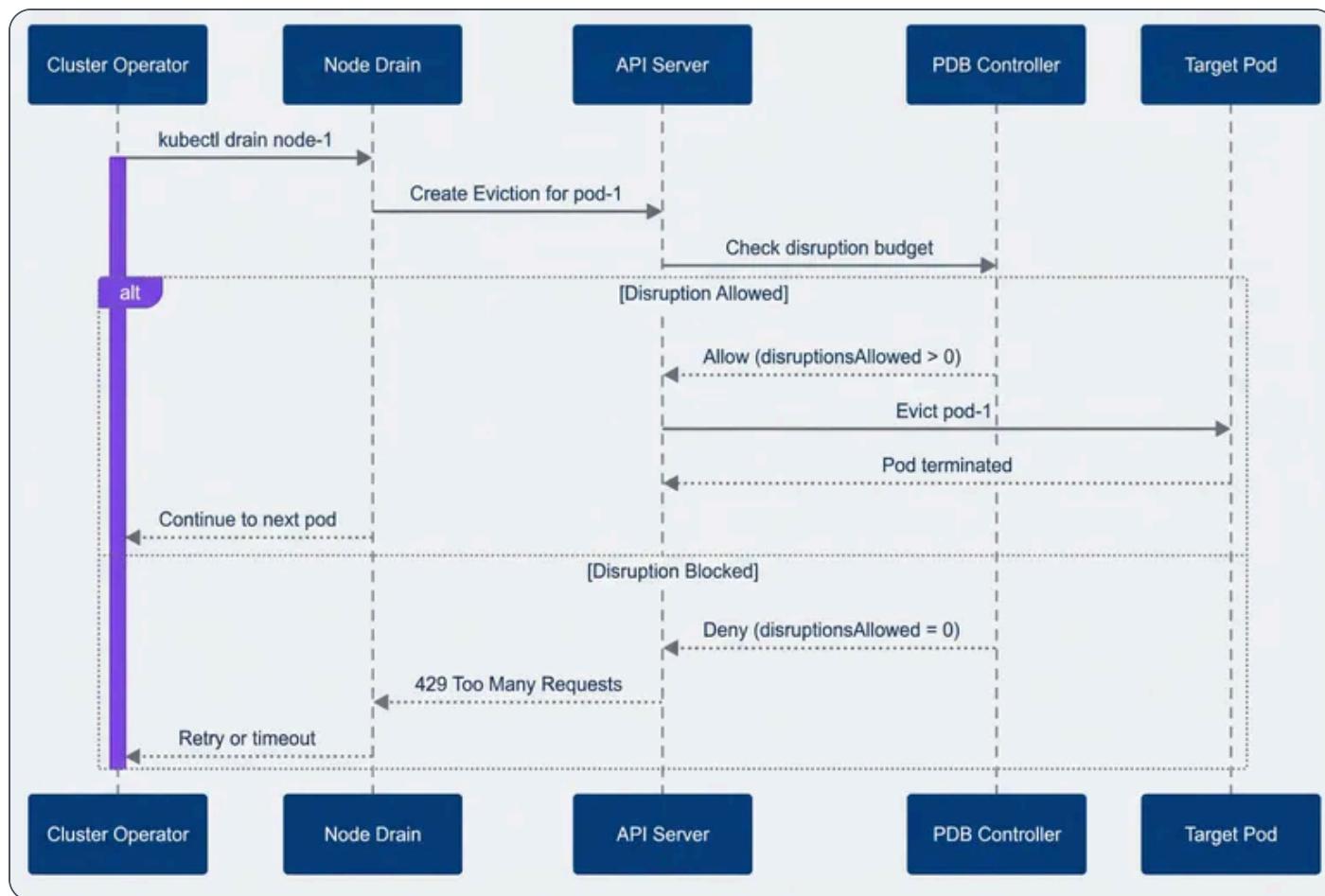
Bash

1	NAME	MIN AVAILABLE	MAX UNAVAILABLE	ALLOWED DISRUPTIONS	AGE
2	api-server-pdb	2	N/A	3	24h

Figure: PDB (Pod Disruption Budget) status showing 3 disruptions allowed (5 healthy pods minus 2 minimum).

The eviction flow works like this: when a drain or autoscaler attempts to evict a pod, it creates an Eviction object through the API (Application Programming Interface) server. The API (Application Programming Interface) server checks the relevant PDB (Pod Disruption Budget). If `disruptionsAllowed` is greater than zero, the eviction proceeds. If it's zero, the API (Application Programming Interface) returns a 429 (Too Many Requests) and the eviction is blocked.





PDB eviction decision flow.

minAvailable vs maxUnavailable

Choosing the wrong mode is one of the most common causes of autoscaler deadlocks. The two PDB (Pod Disruption Budget) modes look similar but behave differently as your replica count changes. Consider a 5-replica deployment:

minAvailable: 2

Means at least 2 must stay up, so 3 can be disrupted simultaneously

maxUnavailable: 1

Means at most 1 can be down, so only 1 can be disrupted at a time



The math gets interesting with percentages. `minAvailable: 80%` on 5 replicas means $\text{ceil}(5 \times 0.8) = 4$ must stay up, allowing only 1 disruption. `maxUnavailable: 25%` means $\text{floor}(5 \times 0.25) = 1$ can be down. Same result, but the behavior diverges as you scale.

#	Configuration	5 Replicas	10 Replicas	Behavior
1	<code>minAvailable: 2</code>	3 can disrupt	8 can disrupt	Absolute minimum
2	<code>minAvailable: 80%</code>	1 can disrupt	2 can disrupt	Scales with replicas
3	<code>maxUnavailable: 1</code>	1 can disrupt	1 can disrupt	Absolute maximum down
4	<code>maxUnavailable: 25%</code>	1 can disrupt	2 can disrupt	Scales with replicas

How PDB configurations scale with replica count.

The dangerous patterns to avoid: setting `minAvailable` equal to your replica count, or setting `maxUnavailable: 0`. Both result in `disruptionsAllowed: 0`, meaning no pods can **ever** be evicted voluntarily. Node drains will hang forever.

INFO

Use `maxUnavailable` for most cases – it’s easier to reason about (“at most 1 pod down at a time”). Use `minAvailable` when you have a hard minimum for quorum-based systems (etcd needs 2 of 3, ZooKeeper needs 2 of 3).

Cluster Autoscaler Interaction

The cluster autoscaler respects PDBs (Pod Disruption Budget) during scale-down operations. Before removing an underutilized node, it checks whether evicting all pods on that node would violate any PDB (Pod Disruption Budget). If it would, the node is skipped. This is usually the right behavior – you don’t want cost optimization to cause outages – but it creates a common operational trap.



Scale-Down and PDBs

Here's the scenario: you have 10 pods spread evenly across 5 nodes (2 pods per node), with a PDB (Pod Disruption Budget) requiring `minAvailable: 9`. The autoscaler identifies node-1 as underutilized and attempts scale-down. But evicting node-1's 2 pods would leave only 8 healthy pods, violating the PDB (Pod Disruption Budget). The autoscaler tries node-2—same problem. Every node is blocked.

The result: you're paying for 5 nodes when 3 would be enough. The PDB (Pod Disruption Budget) that protects availability is now costing you money.

Avoiding Autoscaler Deadlocks

The deadlock happens when your PDB (Pod Disruption Budget)'s `disruptionsAllowed` is smaller than the maximum pods on any single node. The fix is straightforward: ensure your PDB (Pod Disruption Budget) allows at least as many disruptions as the most pods you'll have on one node.

INFO

The autoscaler-safe formula: `maxUnavailable >= ceil(total_pods / node_count)`

For 30 pods across 10 nodes (3 pods per node), you need `maxUnavailable: 4` or higher. With a 20% buffer for scheduling delays during drain, `maxUnavailable: 5` is safer.

autoscaler-safe-pdb.yaml

```
1  apiVersion: policy/v1
2  kind: PodDisruptionBudget
3  metadata:
4    name: autoscaler-friendly-pdb
5  spec:
6    selector:
7      matchLabels:
8        app: my-service
9    # 20% of pods can be unavailable
10   # For 30 pods: floor(30 * 0.2) = 6 can be down
```



```
11     # Safely covers draining any single node (3 pods)
12     maxUnavailable: "20%"
```

PDB (Pod Disruption Budget) configured to allow autoscaler scale-down.

The percentage approach scales automatically as your deployment grows. If you scale to 60 pods across 20 nodes, 20% still allows 12 disruptions – more than enough to drain any node.

⚠ WARNING

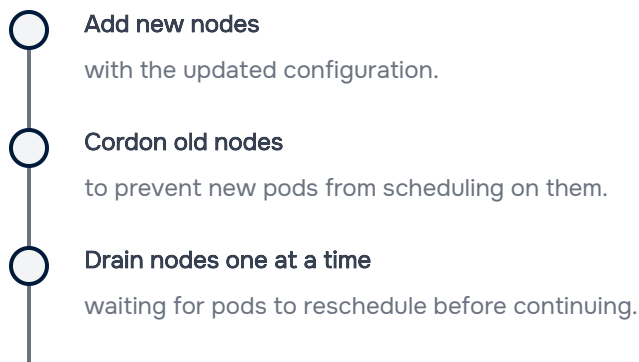
The most common PDB (Pod Disruption Budget) deadlock: you have N nodes with M pods spread evenly, and your PDB (Pod Disruption Budget) requires more than $(total_pods - pods_per_node)$ to be available. Calculate your PDB (Pod Disruption Budget) settings based on your cluster topology, not just your application needs.

Node Rotation Strategies

Node rotations happen for many reasons: security patches, Kubernetes version upgrades, instance type changes, or AMI updates. The key to smooth rotations is working **with** PDBs (Pod Disruption Budget) rather than fighting them.

Rolling Node Updates

The recommended approach is surge-then-drain: add new nodes first, then drain old ones sequentially. This maintains capacity throughout the rotation and gives PDBs (Pod Disruption Budget) room to work.



- Remove drained nodes
once empty.

This sequence maintains capacity throughout the rotation. Cordoning doesn't evict anything – it just marks nodes as unschedulable. Step 3 is where PDBs (Pod Disruption Budget) do their job, ensuring availability while pods migrate. If you're using managed node groups, the autoscaler handles step 4 automatically.

The drain command respects PDBs (Pod Disruption Budget) by default:

Bash

```
1 kubectl drain node-1 \  
2   --delete-emptydir-data \  
3   --ignore-daemonsets \  
4   --timeout=600s
```

Basic drain command with common flags.

The `--timeout` flag is critical. Without it, a blocked drain will hang forever. With it, you'll get an error after 10 minutes that tells you which PDB (Pod Disruption Budget) is blocking.

Handling Stuck Drains

When a drain times out, the first step is identifying what's blocking it:

Bash

```
1 # Check which PDBs have zero disruptions allowed  
2 kubectl get pdb -A -o wide | grep "0"  
3  
4 # Find pods still on the draining node  
5 kubectl get pods -A --field-selector spec.nodeName=node-1
```

Diagnosing a stuck drain.

You're looking for PDBs (Pod Disruption Budget) where the `ALLOWED DISRUPTIONS` column shows `0` – those are the ones blocking your drain. Cross-reference with the pods still on the node to identify the culprit.



Once you've identified the blocking PDB (Pod Disruption Budget), you have several options depending on the situation:

Wait for pods to become healthy

If `disruptionsAllowed` is zero because pods are failing health checks, fix the health issue. Once pods are healthy, the PDB will allow disruptions again.



Scale up the deployment

If you have `minAvailable: 3` but only 3 replicas and one is unhealthy, add a fourth replica. Once it's ready, you'll have headroom to drain.



Temporarily relax the PDB

For planned maintenance windows, you can patch the PDB to be less strict:



Bash

```
1  #!/bin/bash
2
3  # Relax PDB for maintenance
4  kubectl patch pdb my-service-pdb -p '{"spec":{"minAvailable":1}}'
5
6  # Perform drain
7  kubectl drain node-1 --delete-emptydir-data --ignore-daemonsets
8
9  # Restore original PDB
10 kubectl patch pdb my-service-pdb -p '{"spec":{"minAvailable":2}}'
```

Temporarily relaxing a PDB (Pod Disruption Budget) for maintenance.

Force delete as last resort

If you're in an emergency and need the node drained now, you can bypass the eviction API entirely with `kubectl delete pod --force --grace-period=0`. This ignores PDBs completely – the pod just disappears.



Symptom	Likely Cause	Resolution
disruptionsAllowed: 0	Too strict PDB	Relax PDB or scale up deployment
Pods not Ready	Health check failures	Fix pod health issues
Drain timeout	PDB blocking + slow startup	Increase timeout, fix startup time
All nodes stuck	Fleet-wide PDB issue	Review PDB strategy

Stuck drain diagnosis guide.

⚠ DANGER

Never use `--force` in production node drains unless you've exhausted all other options and accepted the availability impact. Forced deletion bypasses PDBs (Pod Disruption Budget) entirely and can cause cascading failures in quorum-based systems.

Production PDB Patterns

After working through the mechanics, let's look at what actually works in production. The right PDB (Pod Disruption Budget) depends on your workload type.

Recommended Configurations

Stateless web services benefit from percentage-based PDBs (Pod Disruption Budget) that scale with your deployment:

stateless-pdb.yaml

```
1  apiVersion: policy/v1
```



```
2  kind: PodDisruptionBudget
3  metadata:
4    name: web-service-pdb
5  spec:
6    selector:
7      matchLabels:
8        app: web-service
9    maxUnavailable: "25%"
```

PDB (Pod Disruption Budget) for stateless services allowing 25% unavailable.

Twenty-five percent is a good default – it balances availability (75% always up) with operational flexibility (multiple pods can be evicted simultaneously during node drains).

Stateful and quorum-based services like etcd, ZooKeeper, or Redis clusters need `minAvailable` set to their quorum size:

quorum-pdb.yaml

```
1  apiVersion: policy/v1
2  kind: PodDisruptionBudget
3  metadata:
4    name: etcd-pdb
5  spec:
6    selector:
7      matchLabels:
8        app: etcd
9    # 3-node cluster needs 2 for quorum
10   minAvailable: 2
```

PDB (Pod Disruption Budget) for a 3-node etcd cluster requiring quorum.

DaemonSets are tricky because there's one pod per node. A percentage-based PDB (Pod Disruption Budget) controls how many nodes can drain simultaneously:

daemonset-pdb.yaml

```
1  apiVersion: policy/v1
```



```
2  kind: PodDisruptionBudget
3  metadata:
4    name: fluentd-pdb
5  spec:
6    selector:
7      matchLabels:
8        app: fluentd
9    # Allow 10% of nodes to drain at once
10   maxUnavailable: "10%"
```

PDB (Pod Disruption Budget) for a logging DaemonSet.

The right percentage depends on your tolerance for gaps. For logging agents like Fluentd, 10% means brief log gaps during rotation – usually acceptable. For monitoring agents or security daemons where gaps are more problematic, use 5%. For non-critical DaemonSets, 20-25% speeds up rotations significantly.

Single-replica deployments

These are a judgment call. A PDB with `maxUnavailable: 1` doesn't prevent disruption – it just makes the eviction use the eviction API instead of direct deletion. This can be useful for visibility and for systems that watch eviction events, but it doesn't provide availability protection. If you need zero downtime, run more replicas.

Batch jobs and CronJobs

Generally shouldn't have PDBs. Jobs should be restartable by design, and a PDB on a job creates operational headaches without providing real protection.

Anti-Patterns to Avoid

Several PDB (Pod Disruption Budget) configurations look reasonable but cause problems:

➤ **minAvailable equals replica count.**

If you have 3 replicas and `minAvailable: 3`, nothing can ever be evicted. This is the most common PDB misconfiguration.

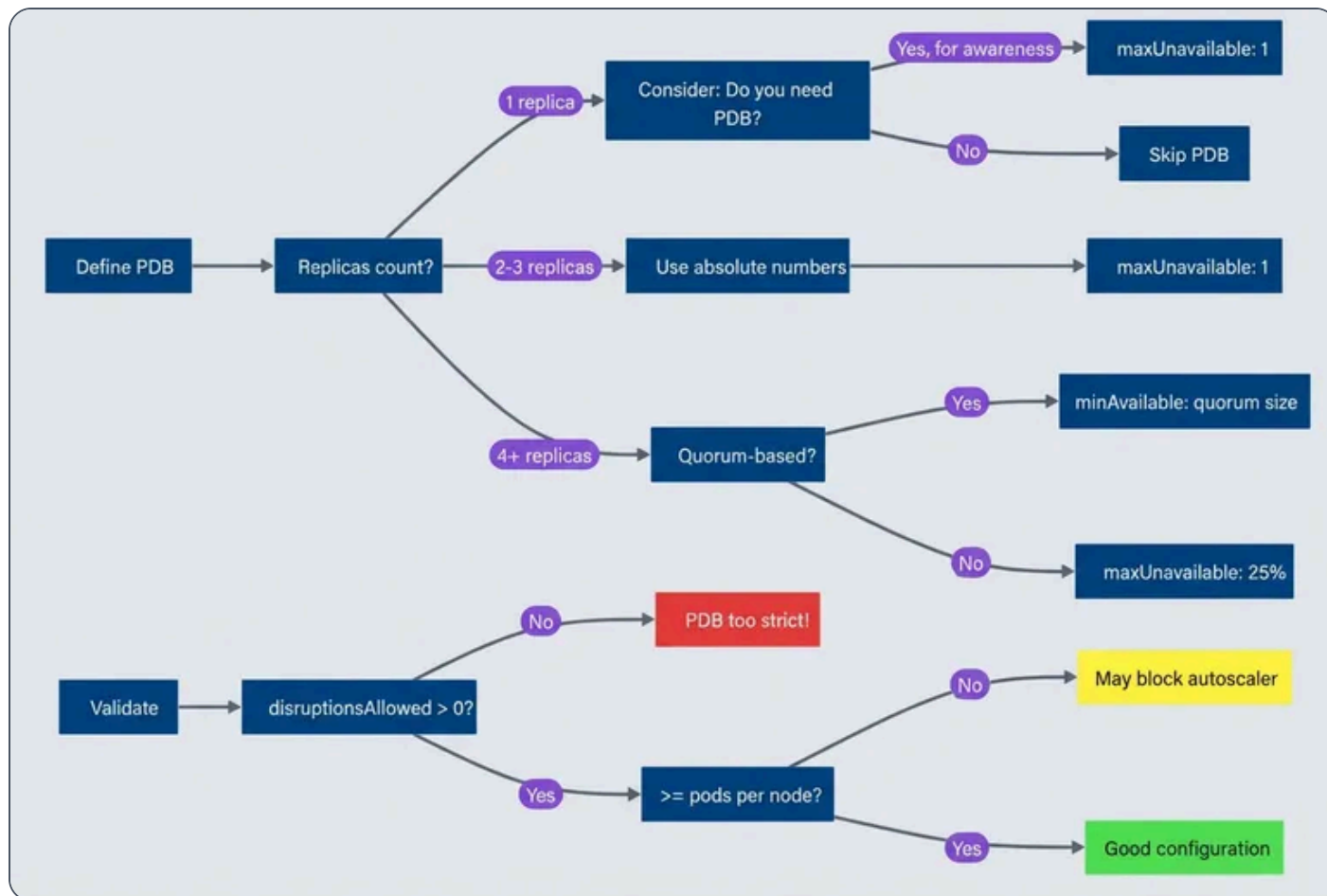
➤ **Percentages that round badly.**

`minAvailable: 90%` with 3 replicas means $\text{ceil}(2.7) = 3$ pods required – blocking all evictions. Use absolute numbers for small deployments.



➤ Multiple overlapping PDBs.

If a pod matches two PDBs, both must allow the disruption. This is stricter than either PDB alone and often catches teams by surprise. For example: Team A creates a PDB for `app: payments` allowing 1 disruption, Team B creates a PDB for `tier: critical` allowing 2 disruptions. Pods with both labels need both PDBs to allow eviction simultaneously – effectively the stricter of the two.



PDB configuration decision tree.

✓ SUCCESS

The golden rule: always have at least 1 more replica than your minAvailable requires, AND ensure maxUnavailable is at least as large as your maximum pods-per-node. This prevents both availability violations and autoscaler deadlocks.



Monitoring and Alerting

PDBs (Pod Disruption Budget) fail silently. A misconfigured PDB (Pod Disruption Budget) doesn't cause an immediate outage – it causes the **next** node rotation to hang at 3 AM. Monitoring catches problems before they block operations.

PDB Health Metrics

The kube-state-metrics project exposes PDB (Pod Disruption Budget) status as Prometheus metrics (ensure kube-state-metrics is deployed and scraped by Prometheus – it's included in most monitoring stacks like kube-prometheus-stack). The key metrics to watch:

- `kube_poddisruptionbudget_status_pod_disruptions_allowed`
How many pods can currently be evicted
- `kube_poddisruptionbudget_status_current_healthy`
Pods currently passing health checks
- `kube_poddisruptionbudget_status_desired_healthy`
Minimum required by the PDB

The most important alert is for PDBs (Pod Disruption Budget) that are blocking all disruptions:

pdb-alerts.yaml

```
1  apiVersion: monitoring.coreos.com/v1
2  kind: PrometheusRule
3  metadata:
4    name: pdb-alerts
5  spec:
6    groups:
7      - name: pdb.rules
8        rules:
9          - alert: PDBBlockingDisruptions
10            expr: kube_poddisruptionbudget_status_pod_disruptions_allowed == 0
11            for: 30m
12            labels:
13              severity: warning
14            annotations:
```



```

15         summary: "PDB {{ $labels.poddisruptionbudget }} blocking all disruptions"
16         description: "PDB in {{ $labels.namespace }} has disruptionsAllowed=0 for
30+ minutes"
17
18     - alert: PDBViolated
19       expr: |
20         kube_poddisruptionbudget_status_current_healthy
21         < kube_poddisruptionbudget_status_desired_healthy
22       for: 5m
23       labels:
24         severity: critical
25       annotations:
26         summary: "PDB {{ $labels.poddisruptionbudget }} violated"
27         description: "Current healthy pods below minimum required"

```

Essential PDB (Pod Disruption Budget) alerts for Prometheus.

The 30-minute threshold for `PDBBlockingDisruptions` balances signal quality with early warning. Brief blocking during deployments is normal; sustained blocking indicates a problem.

The `PDBViolated` alert is more urgent – it means you’re already below your availability target, likely due to pod failures unrelated to eviction.

Health Thresholds

Use these thresholds to assess PDB (Pod Disruption Budget) health at a glance:

Metric	Healthy	Warning	Critical
disruptionsAllowed	> 20% of pods	1-20% of pods	0
currentHealthy vs desired	Equal	---	Below
Blocking duration	< 5 min	5-30 min	> 30 min
Flexibility ratio	> 25%	10-25%	< 10%

PDB health thresholds for monitoring.



The flexibility ratio (disruptions allowed divided by total pods) tells you whether a PDB (Pod Disruption Budget) will block autoscaler scale-down. Below 20%, you're likely to hit deadlocks.

INFO

Alert on PDBs (Pod Disruption Budget) blocking disruptions for more than 30 minutes – this indicates either a misconfigured PDB (Pod Disruption Budget) or unhealthy pods. Either way, it needs human attention before it blocks the next operational event.

Conclusion

PDBs (Pod Disruption Budget) are contracts between workload owners and platform operators. The goal is controlled disruption, not zero disruption.

Configure PDBs (Pod Disruption Budget) to allow at least enough disruptions for single-node drains – that's the minimum for cluster operations to work. Monitor for PDBs (Pod Disruption Budget) that block disruptions, and have clear procedures for resolving stuck drains when they happen.

SUCCESS

If your platform team is constantly battling PDBs (Pod Disruption Budget) during node rotations, the PDBs (Pod Disruption Budget) are misconfigured – not too loose, but too strict.

A cluster that can't be maintained isn't a reliable cluster. PDBs (Pod Disruption Budget) that block security patches, version upgrades, and capacity optimization aren't protecting availability – they're trading one kind of risk for another. The best PDB (Pod Disruption Budget) configuration is invisible: it protects your workloads during operations without anyone noticing. That 6 AM page about stuck nodes? With proper PDB (Pod Disruption Budget) configuration, it doesn't happen.



Copyright © 2024 Webstack Builders, Inc.

The text, diagrams, and images in this work are licensed under CC BY-NC 4.0

All code samples in this article are licensed under the MIT License. Feel free to use, modify, and distribute them in any project.

<https://www.webstackbuilders.com/articles/kubernetes-pod-disruption-budget-autoscaler-node-rotation>

